
LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT)

Modul 4-5



Oleh :

Nama : Rena Anggraini

NPM : 25782022

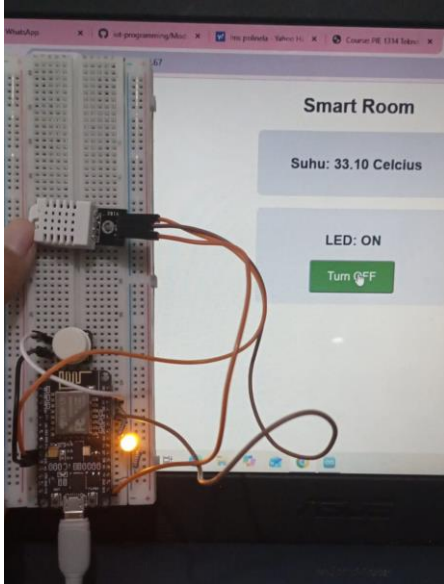
Kelas : TRI (3A)

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, A.Md

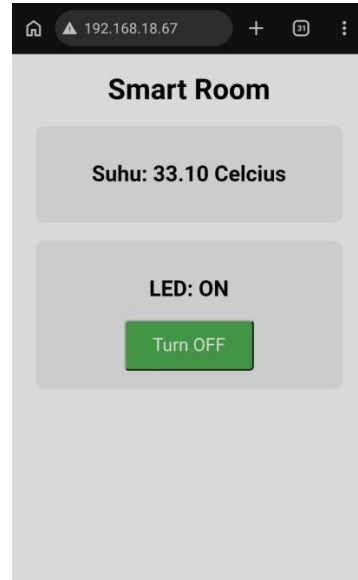
JURUSAN TEKNOLOGI INFORMASI
TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG
TAHUN 2026

PRAKTIKUM 1

Real-Time Dashboard & Sinkronisasi Tombol Fisik



Hasil Foto Perangkat



Hasil Web suhu

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' )
Client WebSocket #0 terhubung
Client WebSocket #6 terputus
Client WebSocket #7 terhubung
Client WebSocket #7 terputus
Client WebSocket #8 terhubung
Client WebSocket #9 terhubung
Client WebSocket #8 terputus
Client WebSocket #9 terputus
Client WebSocket #10 terhubung
```

Hasil Serial Monitor

4.6 Latihan

1. **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi WebSockets jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode Meta Refresh HTTP HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab:

- **Efisiensi Bandwidth:** HTTP standar bersifat Request-Response dengan koneksi yang terputus-putus, di mana setiap pembaruan mengharuskan browser mengunduh ulang seluruh kerangka halaman web (HTML, CSS, skrip, gambar) beserta header HTTP yang berukuran besar. Sebaliknya, WebSockets menggunakan satu koneksi terbuka yang persisten dan hanya mengirimkan paket data kecil berformat JSON (misalnya nilai sensor atau status LED saja).
 - **Tidak Berkedip (No Flickering):** Meta Refresh atau refresh manual memuat ulang seluruh halaman secara total, menyebabkan layar berkedip putih sesaat. Dengan WebSockets, pembaruan dilakukan secara real-time menggunakan manipulasi DOM (Document Object Model) pada elemen spesifik di halaman web tanpa memuat ulang kerangka HTML.
2. **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets (reaksi antara browser dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab:

- **Pembekuan Sistem (Blocking):** Fungsi `delay()` bersifat memblokir (blocking), yang berarti CPU NodeMCU akan berhenti total dan tidak mengeksekusi instruksi lain selama 3 detik.
- **Dampak pada WebSockets:** Selama proses delay tersebut, mikrokontroler tidak dapat mendeteksi penekanan tombol fisik secara instan, tidak dapat membersihkan klien (`ws.cleanupClients()`), serta mengalami penundaan respons jaringan. Jika koneksi atau data masuk terjadi saat CPU sedang diam, hal ini dapat menyebabkan timeout, kegagalan respons asinkron, hingga memicu

watchdog timer yang berujung pada Crash/Restart otomatis pada ESP8266.

3. **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh script yang ditulis oleh Anda?

Jawab:

- **tempValue:** Target elemen `<span id="tempValue">` untuk menyuntikkan dan memperbarui data angka suhu secara dinamis.
 - **ledStatus** dan **toggleBtn:** Target elemen untuk mengubah status teks indikator LED (menjadi "ON" atau "OFF"), serta mengubah kelas warna dan teks tombol aksi secara langsung (DOM Manipulation).
4. **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (temperature). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (humidity) secara real-time via WebSockets:
- a. Sisi Back-End (C++): Buat variabel penampung (misal: `String currentHum = "--"`);. Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer non-blocking `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke browser (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
 - b. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:

- Sisi Back-End (C++):
Tambahkan variabel penampung global,
String currentHum = "--";
- Perbarui fungsi notifyClients() untuk menyertakan data kelembapan ke dalam format JSON

```
void notifyClients() {  
    String jsonString = "{\"led\": \"\" + String(ledState ? 1 : 0) + "\", \"  
    jsonString += \"suhu\": \"\" + currentTemp + "\", \"  
    jsonString += \"hum\": \"\" + currentHum + \"}\"";  
    ws.textAll(jsonString);  
}
```

- Tambahkan kartu penampil kelembapan pada bagian <body> HTML

```
<div class="card">  
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>  
</div>
```

- Tambahkan penanganan data hum pada fungsi onMessage(event) di dalam blok JavaScript

```
if(dataObj.hum !== undefined) {  
    document.getElementById('humValue').innerHTML = dataObj.hum;  
}
```

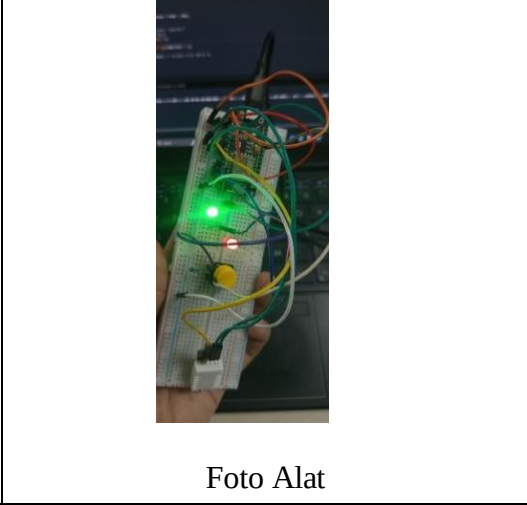
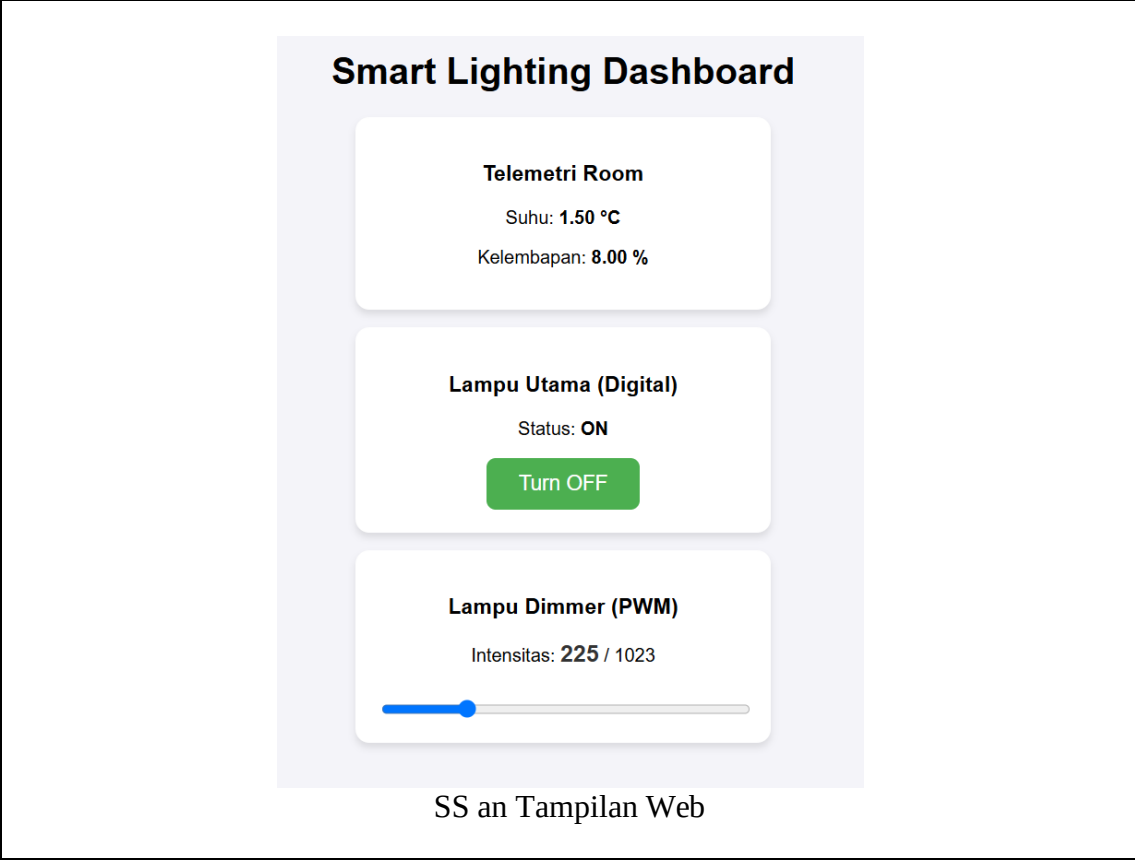
[illegible]

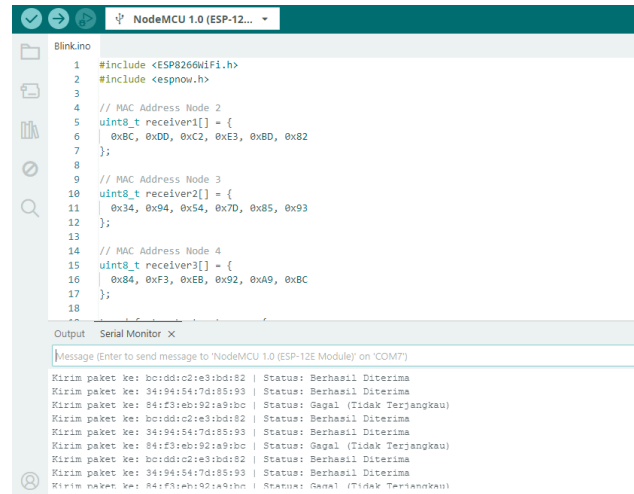
Foto Alat



SS an Tampilan Web

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



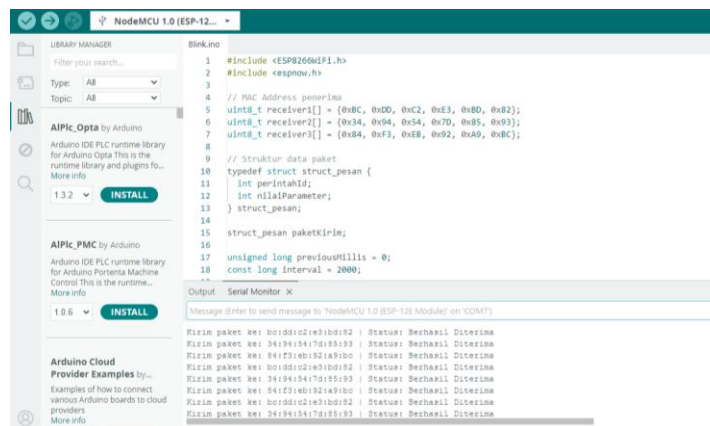
```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // MAC Address Node 2
5 uint8_t receiver1[] = {
6   0xBC, 0xDD, 0xC2, 0xE3, 0x8D, 0x82
7 };
8
9 // MAC Address Node 3
10 uint8_t receiver2[] = {
11   0x34, 0x94, 0x54, 0x7D, 0x85, 0x93
12 };
13
14 // MAC Address Node 4
15 uint8_t receiver3[] = {
16   0x84, 0xF3, 0xEB, 0x92, 0xA9, 0xBC
17 };
18
```

Output Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:bc | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:bc | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:bc | Status: Gagal (Tidak Terjangkau)

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)



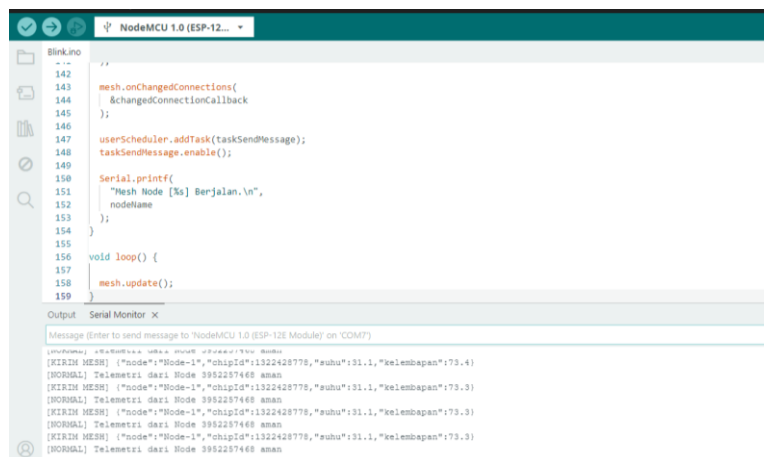
```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // MAC Address penerima
5 uint8_t receiver1[] = {0xBC, 0xDD, 0xC2, 0xE3, 0x8D, 0x82};
6 uint8_t receiver2[] = {0x34, 0x94, 0x54, 0x7D, 0x85, 0x93};
7 uint8_t receiver3[] = {0x84, 0xF3, 0xEB, 0x92, 0xA9, 0xBC};
8
9 // Struktur data paket
10 typedef struct struct_pesan {
11   int perintah;
12   int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long Interval = 2000;
19
```

Output Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:bc | Status: Berhasil Diterima
Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:92:a9:bc | Status: Berhasil Diterima
Kirim paket ke: b0:dd:c2:e3:bd:92 | Status: Berhasil Diterima
Kirim paket ke: 34:94:54:7d:85:93 | Status: Berhasil Diterima

5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)



```
142 //
143 mesh.onChangedConnections(
144   &changedConnectionCallback
145 );
146
147 usersScheduler.addTask(taskSendMessage);
148 taskSendMessage.enable();
149
150 Serial.printf(
151   "Mesh Node [%s] Berjalan.\n",
152   nodeName
153 );
154 }
155
156 void loop() {
157   mesh.update();
158 }
159
```

Output Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

[KIRIM MESH] ("node":"Node-1", "chipId":1322428778, "subu":31.1, "kelembapan":79.4)
[NORMAL] Telemetri dari Node 3952257468 aman
[KIRIM MESH] ("node":"Node-1", "chipId":1322428778, "subu":31.1, "kelembapan":79.3)
[NORMAL] Telemetri dari Node 3952257468 aman
[KIRIM MESH] ("node":"Node-1", "chipId":1322428778, "subu":31.1, "kelembapan":79.3)
[NORMAL] Telemetri dari Node 3952257468 aman
[KIRIM MESH] ("node":"Node-1", "chipId":1322428778, "subu":31.1, "kelembapan":79.3)
[NORMAL] Telemetri dari Node 3952257468 aman

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

```
NodeMCU 1.0 (ESP-12...)
Blink.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh"
6 #define MESH_PASSWORD "Iotmeshpassword"
7 #define MESH_PORT 5555
8
9 #define DHTPIN 2
10 #define DHTTYPE DHT22
11
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Nama Node 1
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

[KIRIM MESH] {"node":"Node-1","chipId":1322426778,"suhu":32.2,"kelembapan":78.1}

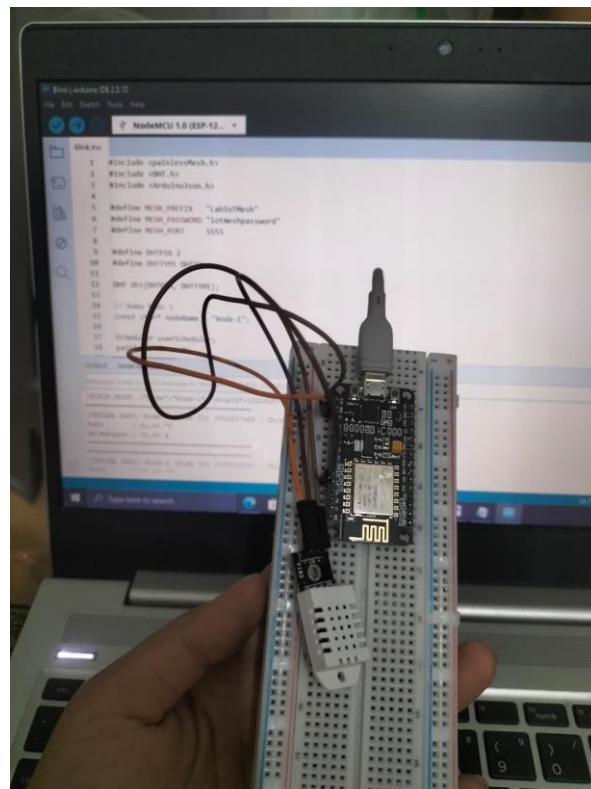
[TERIMA DARI] Node-4 (Node ID: 3952257468 | Chip ID: 3952257468)

Suhu : 31.40 °C

Kelembapan : 74.90 %

[TERIMA DARI] Node-2 (Node ID: 3269705090 | Chip ID: 3269705090)

Suhu : 31.30 °C



5.6 Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):**
Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - a. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
 - b. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
 - c. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawab:

- **Ketiadaan Infrastruktur & Biaya:** Di area perkebunan luas, pemasangan kabel atau penyediaan router fisik sangat mahal, tidak praktis, dan sering kali tidak tersedia sumber listrik utama. ESP-NOW dan *painlessMesh* memungkinkan komunikasi nirkabel mandiri tanpa infrastruktur pusat.
 - **Menghindari Titik Kegagalan Tunggal (Single Point of Failure):** Pada jaringan terpusat ber-router, jika router utama mati atau tersambar petir, seluruh sistem sensor lumpuh total. Pada jaringan Mesh, rute data bersifat dinamis (*self-healing*); jika satu node perantara mati, paket data otomatis dialihkan melalui jalur tetangga lain.
 - **Efisiensi Daya & Kecepatan (Low Power & Latency):** ESP-NOW tidak memerlukan proses handshake Wi-Fi yang lama (WPA2/DHCP), sehingga mikrokontroler dapat mengirim data dengan cepat dalam hitungan milidetik dan kembali tidur lelap (*deep sleep*) guna menghemat baterai.
-
- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):**

Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi.

- i. Tekan tombol fisik RST atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh callback `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawab:

- **Korupsi Memori (Memory Corruption / Garbage Values):** ESP-NOW mentransmisikan data mentah berupa kumpulan bita (raw bytes) melalui pointer memori. Fungsi `memcpy()` di sisi penerima akan langsung memetakan bita-bita tersebut ke dalam variabel penampung berdasarkan struktur posisinya.
 - **Hasil Salah atau Crash:** Jika urutan atau tipe data tidak identik, pembacaan nilai variabel akan meleset (misalnya nilai sensor suhu masuk ke variabel ID perintah), yang berpotensi menghasilkan data cacat, logika program salah, atau memicu *exception/crash* pada memori mikrokontroler akibat salah pembacaan tipe data pointer.
-
- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai Pengawas Kondisi Suhu Kritis: program HANYA mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai di atas 32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
 - ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.

iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawab:

- **Bebas Konfigurasi MAC Address:** Berbeda dengan ESP-NOW standar yang mewajibkan pengembang mencatat dan mendefinisikan alamat MAC fisik lawan secara statis di dalam kode, `painlessMesh` menggunakan nama pengenalan jaringan (Mesh Prefix) dan kata sandi bersama (Mesh Password) secara otonom.
- **Mekanisme Estafet (Multi-Hop Relay):** Ketika jarak antar-node terlalu jauh atau terhalang rintangan (> 100 meter), node perantara di tengah bertindak sebagai repeater yang menerima dan meneruskan (relai) paket data secara berantai dari satu node ke node lainnya hingga sampai ke tujuan akhir

```
void receivedCallback(uint32_t from, String &msg) {
    // Cetak informasi dasar pengirim dan isi pesan mentah ke Serial Monitor
    Serial.printf("Pesan diterima dari Node [%u]: %s\n", from, msg.c_str());

    // Alokasi memori untuk parsing JSON (pastikan ukuran cukup, misal 1024 byte)
    DynamicJsonDocument jsonDoc(1024);
    DeserializationError error = deserializeJson(jsonDoc, msg);

    // Periksa apakah pesan yang masuk valid berformat JSON
    if (error) {
        Serial.print("Gagal melakukan parsing JSON: ");
        Serial.println(error.c_str());
        return;
    }

    // Ekstraksi data spesifik dari payload JSON (Contoh: pembacaan sensor/status)
    if (jsonDoc.containsKey("suhu") && jsonDoc.containsKey("hum")) {
        float sensorSuhu = jsonDoc["suhu"];
        float sensorHum = jsonDoc["hum"];

        Serial.println("--- Data Telemetry Mesh Terdeteksi ---");
        Serial.printf("> Suhu      : %.2f \"C\n", sensorSuhu);
        Serial.printf("> Kelembapan: %.2f %\n", sensorHum);
    }

    // Contoh penanganan perintah kontrol (misal perintah LED dari node lain/Master)
    if (jsonDoc.containsKey("command")) {
        String cmd = jsonDoc["command"];
        if (cmd == "LED_ON") {
            digitalWrite(LED_BUILTIN, LOW); // Nyalakan LED (Active Low pada ESP8266/NodeM)
            Serial.println("> Perintah diterima: LED dihidupkan.");
        } else if (cmd == "LED_OFF") {
            digitalWrite(LED_BUILTIN, HIGH); // Matikan LED
            Serial.println("> Perintah diterima: LED dimatikan.");
        }
    }
}
```